

Pre-Entry Preparation Assignment Deliverables: ”AISIMS Data Model – Written Deliverables”

AISIMS & AIVISTA

Assignee: Daffa

Deliverable Period: Week 3

1. Data Model – Training Proposal

a. Entity Relationships Diagram (ERD)

(The section in this document discusses the final result of the design, the full breakdown, decision reasoning and AI-assisted work log is discussed in the Working Log document)

An ERD is composed of two components: entities and relationships. The entities in the ERD draft were built directly from the data hierarchy listed in the pack (project, building, floor, zone, and so on). The relationships, however, required deeper design work, discussed later in this document, since they are a product of the design decisions made around the identity model, revision handling, and even input design (although not built out in full for the Week 3 scope).

Settling this design mattered upfront because it dictates the relationships in the data model as well as which additional “helper” entities are needed beyond what the hierarchy alone implies. For example, the decision to keep the identities entity separate from *objects* entity (previously referred to as the *members* entity) stems directly from the workflow model for data input and revisions. Although discussed first in this document, the actual work sequence for this deliverable was completed after the revision design had mostly taken shape.

The proposed data model follows two groups of data: *Project Configuration* and *Project Design*. Each group corresponds to a different kind of table role, a dimension table or a fact table. *Project Configuration* consists of reference entities such as projects, buildings, materials, zones, floors, and sections (previously referred to as “dimensions,” renamed to avoid confusion with the database concept of a dimension table), which feed into the fact tables that make up *Project Design*: the list of objects and their reinforcement, plus quantity as a derived output of the object data. This distinction forms the basis for how input and revision behavior differ between the two groups, discussed further in the revision model design decisions.

The ERD is represented in DBML using dbdiagram, organized so that it reflects the data workflow starting from the higher hierarchy, grouped by whether an entity belongs to project configuration or project design, and ending with the output entities.

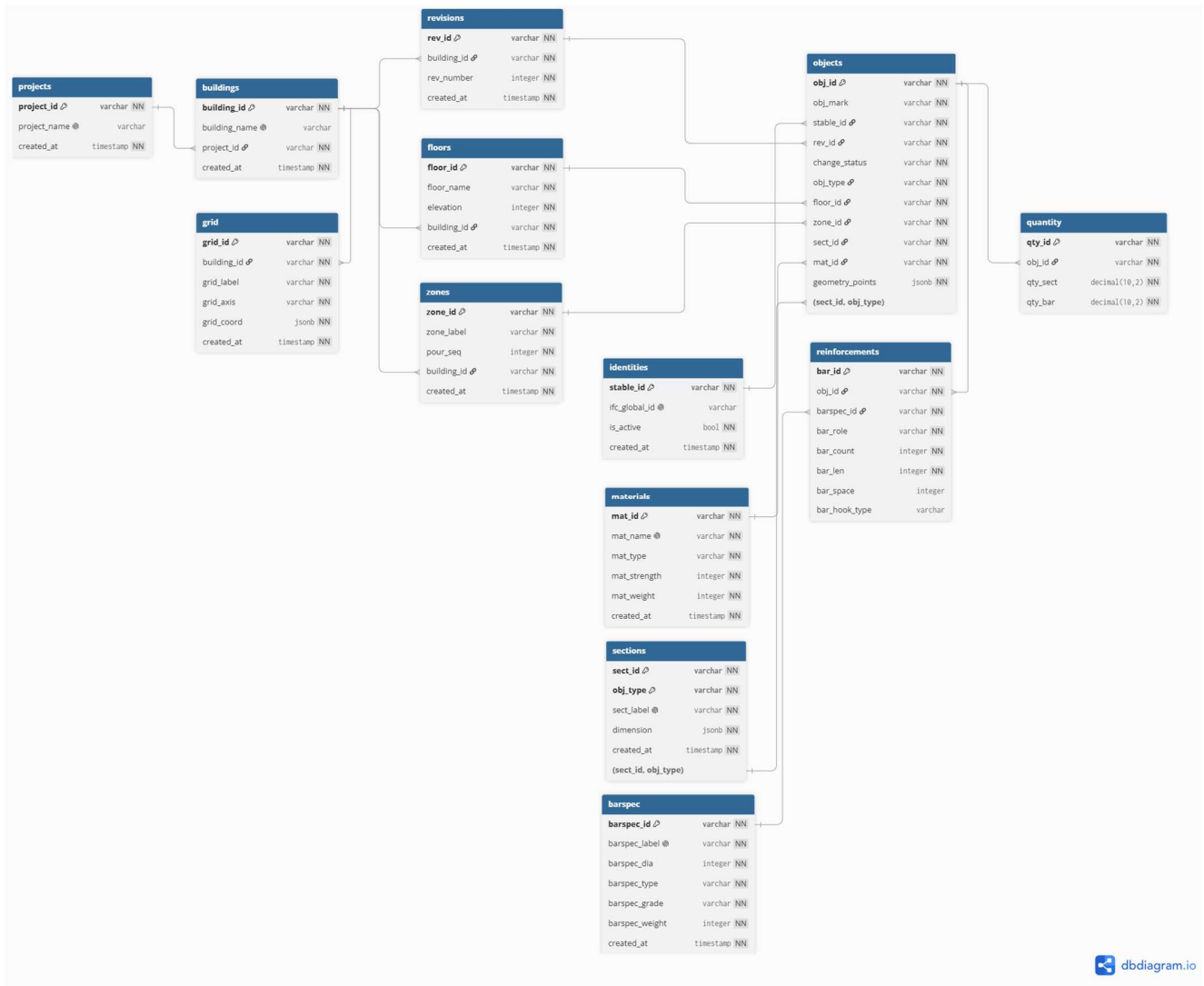


Figure 1. Entity Relationship Diagram

b. Data Dictionary and Key Relationships

(The section in this document discusses the final result of the design, the full breakdown, decision reasoning and AI-assisted work log is discussed in the Working Log document)

The data dictionary in **Appendix A** covers the full explanation and relationship for the entire thirteen entities in total, grouped into three roles: *Project Configuration* (projects, buildings, floors, zones, materials, sections, and barspec), *Project Design* (revisions, identities, objects, and reinforcement), and derive *output* (quantity). One entity sits outside this pattern: grid is a configuration entity that is not related to objects at all. It exists solely to support drawing generation later, and a member's actual location is still determined entirely through its geometry points, independent of the grid. The full table is intentionally placed as an appendix to not disrupt the reading flow.

Key relationships derived from the data dictionary can be categorized as follows:

Table 1. Key Relationships

One-to-Many (Project Configuration ownership hierarchy)

From	→	To	Field
projects	one-to-many	buildings	project_id
buildings	one-to-many	floors	building_id
buildings	one-to-many	zones	building_id
buildings	one-to-many	grid	building_id
buildings	one-to-many	revisions	building_id

Reusable-Reference (Object borrows from Project Configuration catalogs)

From		To	Field
objects	many-to-one	floors	floor_id
objects	many-to-one	zones	zone_id
objects	many-to-one	sections	sect_id + obj_type (composite)
objects	many-to-one	materials	mat_id
reinforcements	many-to-one	barspec	barspec_id

Revision Relationship (snapshot / identity mechanism)

From		To	Field
objects	many-to-one	revisions	rev_id
objects	many-to-one	identities	stable_id
reinforcements	many-to-one	objects	obj_id
quantity	many-to-one	objects	obj_id

2. Design Decisions – Training Proposal

a. Geometry Representations

The geometry representation design was initially modeled after how ETABS describes “start and end” geometry under member properties, using x, y, z coordinates. This approach was chosen because any point in three-dimensional space can be uniquely and unambiguously located using exactly three coordinate values, and the approach already has a proven precedent in analysis software. Adopting the same mental model reduces the translation effort between a structural analysis package and AISIMS' own data model, which matters because the Ingestion stage is described as a deterministic, rule-based reading. A geometry format that engineers already recognize is easier to validate by eye and less likely to introduce misinterpretation during that translation.

Beams and columns are treated as line elements represented by two points, since both are prismatic members: straight, with a constant cross-section along their length, while the cross-section itself is deliberately kept separate in the section entity rather than embedded in the geometry. This separation keeps the geometry model focused purely on spatial position and lets the same cross-section be reused across many members without repeating shape data. Shell objects such as walls and slabs are represented as an ordered set of boundary points that enclose a polygonal area, in the same way a floor outline is traced point by point. This approach does not currently accommodate curved members or dome

structures; one modeling guideline in ETABS states that curved members should be modeled as a series of short straight segments instead, but this approach is still left as an open decision outside AICONS' system. Furthermore, stairs and footings are deliberately left out of this week's scope, as the approach for representing these two object types remains an open question.

In this design proposal, all coordinates are anchored to a single global axis for simplicity, since one endpoint-based model is designed to represent a single building with its own deterministic (0, 0, 0) origin; it cannot represent a complex of multiple buildings, where local and global axes would need to be reconciled separately. Because every *building_id* is examined independently, global axis and local axis are treated as identical by design, so a member's position is immediately legible without requiring a separate reference to interpret it. This simplification would not hold in a more complex scenario where a single model represents several buildings at once and a given member's position must be resolved relative to which building it belongs to, which side, and which orientation; that scenario falls outside this week's scope.

b. Identification Model

(The section in this document discusses the final result of the design, the full breakdown, decision reasoning and AI-assisted work log is discussed in the Working Log document)

It has been understood that there is a meaningful difference between a “readable” identifier and a “traceable” identifier. What was synthesized in Week 2 was the former: an engineering identifier, currently agreed to be referred to as the **Member / Object Mark** (for example, “C1.02.001” – completely up to the engineer’s input convention), which can be easily interpreted by engineers working in the field as one code representing the section, floor, and grid position of a member. This readable identifier, however, carries a traceability problem, since a single member can end up with a changing identity as its attributes change from one revision to the next.

To address this, a stable, non-changing identity is needed to represent a single member across every revision throughout the life of the project. This is referred to as the **Stable ID**: a single, non-duplicate identifier belonging to one member for the project's entire lifespan. It is designed as an internal system identifier, one that does not depend on any external source or representing the physical attribute to name a member. On that same logic of a stable identifier, when the data originates from an external source, IFC addresses the same concern through its **Global IFC ID**. As long as a member is not deleted, its changing attributes can still be traced using this same identifier. As an addition, another auto-generated **Database ID** *obj_id* is needed for database row handling; this, however, does not concern itself with traceability or readability, since it exists solely to represent a member's row for one specific revision inside the database.

Table 2. Object / Member Identifier Types

Identifier	Field Names	Assigned By	Changes Across Revisions?	Example
Member / Object Mark	obj_mark	Engineering convention (human-input)	Yes — may change when section or location changes	C1.F02.001
Stable ID	stable_id	Internal system (AICONS)	No — remains fixed for the life of the member	Internal value, not human-facing
Global IFC ID	ifc_global_id	External BIM software	No — remains fixed as long as the member is not deleted	External value, present only from IFC-sourced data

Identifier	Field Names	Assigned By	Changes Across Revisions?	Example
Database ID	obj_id	Codebase (system-generated)	No — remains fixed for its respective revision, will append with another entry on each occurrence of revision	MBR-001

One assumption worth stating explicitly, for cases where the Global IFC ID may not be present, is that there is no way to determine the traceability of a member that has changed completely (for instance, relocated by 50 mm and changed in section at the same time) without human input confirming that it is in fact the same member. The location has changed and the attributes have changed, yet there remains a possibility that it still represents the same underlying member. Without an external IFC source confirming that the relocated object is the same object, this cannot be determined automatically. Therefore, the current proposed design stated the Stable ID is assigned by a human whenever an IFC identifier is absent.

This identity definition becomes crucial in designing the revision handling model. How a member is determined to be changed, added, unchanged, deleted, or even restored between revisions is keyed on this stable identifier. Detecting that a member has changed always requires comparing two different states belonging to the same subject. That subject needs a fixed, unchanging reference for the comparison to carry any meaning, and that reference is the Stable ID. As an example, a beam whose section changes (say from a rectangular to a circular profile) would see its mark changes accordingly (as illustrated in the Week 2 example, from C1 to C2), while its Stable ID remains the same. The same logic applies to a relocated column: if a new grid line is added between A and B to create grid AA, the mark may change from A1 to AA1, while the Stable ID and its associated IFC identifier, if present, remain unchanged. It should be acknowledged that a column's as-built position does not always sit exactly on its intended grid line; how AICONS handles member mark generation in that situation remains out of scope for this week.

Table 3. Chronological Example for a Changed Object

stable_id	ifc_global_id	obj_id	obj_mark	rev_id	change_status	Explanation
12345-abcd	123456789\$	OBJ-001	C1.A1.001	REV-000	<i>added</i>	Initial version of the member
12345-abcd	123456789\$	OBJ-002	C1.AA1.001	REV-001	<i>changed</i>	The same member (same stable IDs). Changed location (grid). The mark is changed following the context
12345-abcd	123456789\$	OBJ-003	C2.AA1.001	REV-002	<i>changed</i>	The same member (same stable IDs). Changed location (grid). The mark is changed following the context

Expanding the identity design beyond structural objects, every entity in the model carries both a database-facing identity and a user-facing identity, detailed further in Appendix A. The database-facing identity are the fields ending in *_id*, and it is always a system auto-generated one, since it must uniquely represent a single row regardless of which project, building, or other location it belongs to. The user-facing identity is the field ending in *_name*, *_label*, or *_mark*, and it exists purely to make an entity readable and identifiable to an engineer; whether it must be unique, and at what scope, depends on the entity itself, as set out in the

data dictionary. A traceable identity is also required for data entries inside the database; in the same way the Stable ID is required for structural objects.

Table 4. Other Entities Identifier Types

Identifier	Field Names	Assigned By	Changes Across Revisions?	Example
Database-facing IDs	*_id	System	Depends — Higher hierarchy data such as project name or building name can be changed, lower hierarchy entries that feeds object attributes such as materials can't be changed since it leads to implicit change to the previous revisions	MAT-001, BAR-001, OBJ-001
User-facing IDs	*_mark *_name *_label	Engineering convention (human-input)	No — remains fixed for the life of the attribute	Internal value, not human-facing

c. Revision Model Design

(The section in this document discusses the final result of the design; the full breakdown, decision reasoning, and AI-assisted work log is discussed in the Working Log document)

Part 1: Revision Nature of Behavior

As stated in the Week 1 supplementary pack, a revision is handled as a snapshot rather than an in-place edit: a new revision is a complete, immutable state of the project data, created from a base revision, while the base revision itself continues to exist unaltered. This snapshot principle forms the baseline for every design decision described below. Two further properties shape how revisions behave. First, a revision is expensive: it should be reviewed and approved before becoming authoritative, so the design borrows the Git convention of a “staging” step, where an engineer can inspect a diff of what has changed before “committing” it as an official revision (understood, not fully designed and built. Currently considered as an upstream process before data is put into the system). Second, a revision history is expected to stay up to date and unambiguous: unlike branching in software development, which in project execution experience tends to create mismanagement and miscommunication, revisions are strictly linear, and the “current” version of a building is always its latest revision. Restoring a previously deleted object still produces a new revision and is recorded explicitly as a “restored” status, rather than allowed to reappear implicitly once its deletion is undone.

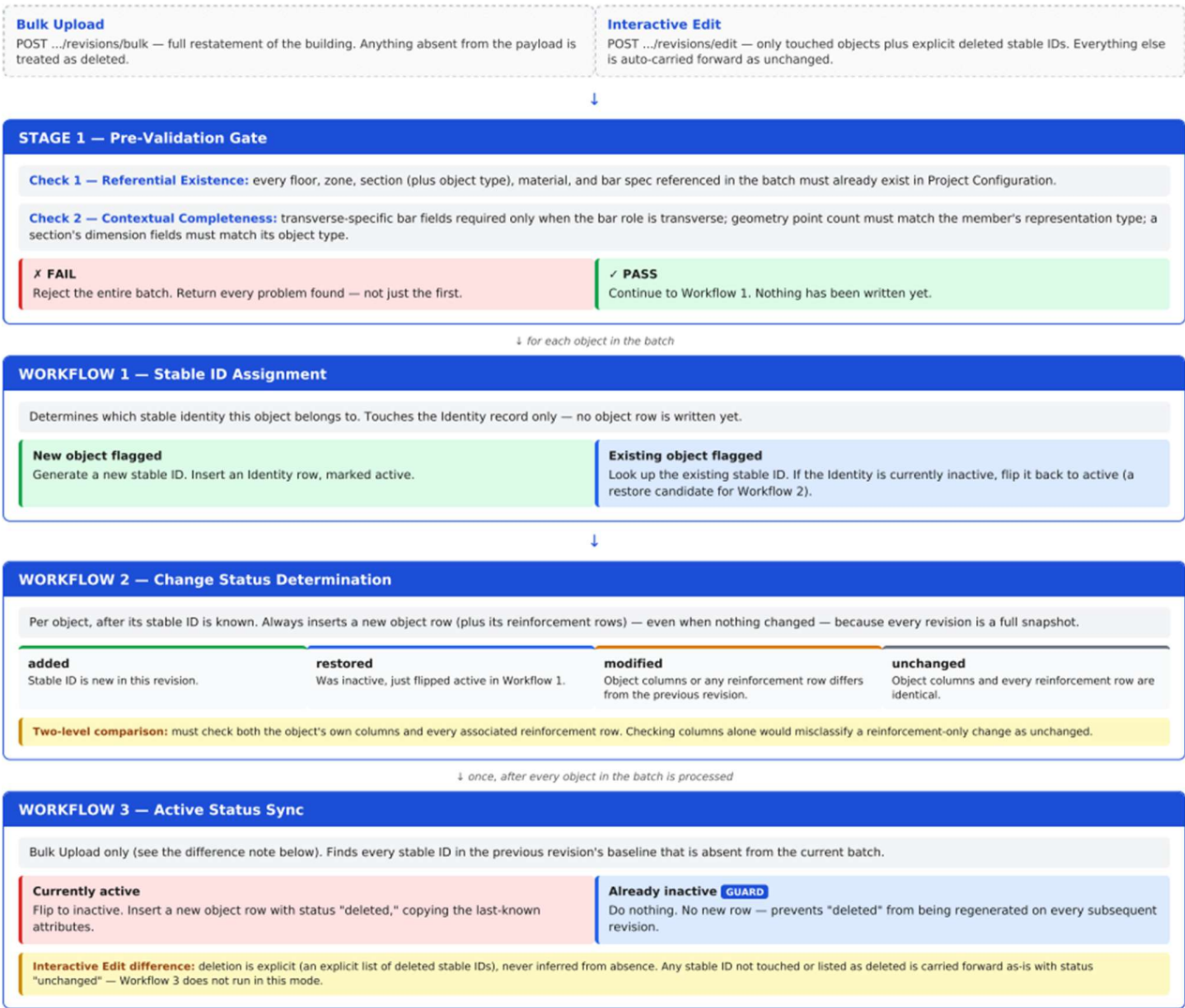
Because the revision model is a full snapshot, any change to even a single object, however small, results in an entirely new revision version representing the whole building, not just the changed object. To keep this meaningful at the granular level, each object additionally carries a change status describing what happened to it specifically within that revision: **added**, **modified**, **unchanged**, **deleted**, or **restored**. The change status is what lets an engineer read a revision as a diff against the previous one, rather than as an undifferentiated wall of data.

A change to a building's data can originate from one of either group described earlier: a change to Project Configuration (a dimension table, for example adding or reassigning a material) or a change to Project Design (a fact table, for example adding, removing, editing attribute of an object). Only a change to project design triggers a new revision; configuration changes are set up independently and do not on their own

create a new snapshot. This distinction is directly tied to how project configuration behaves once it is referenced. A configuration item, once referenced by an object, is treated as append-only: editing it in place would implicitly change every object that already refers to it, silently altering revisions that were already approved. A configuration change is therefore only ever made by adding a new row, never by overwriting an existing one, and it only becomes part of a building's actual history once a project design change, an object referencing it, actually occurs. This is why only a change to project design, not project configuration on its own, is what triggers a new revision.

Part 2: Revision Workflows

Processing a revision submission through input follows four sequential steps.



as *unchanged*, since its absence from the payload does not, by itself, mean anything. This difference in how deletion is inferred, implicitly from absence in *Bulk Upload*, versus explicitly declared in *Interactive Edit*.

The first step is a **Pre-Validation Gate**, which checks the entire submitted batch before any data is written. It verifies that every configuration item referenced in the submission already exists (*Referential Existence*), and that required fields are present given the context of each entry (*Contextual Completeness*). If any check fails, the entire batch is rejected together with a complete list of every problem found, rather than stopping at the first error.

The second step, **Stable ID Assignment**, runs per object once the submission has passed the pre-validation gate. It determines which stable identity an incoming row belongs to: if the object is flagged as new (*is_new=true*), a new Stable ID is generated and a new Identity row is created, marked active. If the object is flagged as existing (*is_new=false*), its Stable ID is looked up instead, and if its Identity is currently inactive, it is flipped back to active, marking it as a restore candidate for the next step.

The third step, **Change Status Determination**, runs per object once its identity is known. Each object is compared against the previous revision's baseline, checking both the object's own attributes and every reinforcement row associated with it, and is classified as **added**, **modified**, **unchanged**, or **restored** accordingly. Comparing at both levels matters, since a reinforcement-only change would otherwise be misclassified as unchanged if only the object's own columns were checked.

The fourth step, **Active Status Sync**, runs once at the end of the batch rather than per object. It identifies any object that existed in the previous revision but is absent from the current submission and marks it as **deleted**, guarded by checking both that the object existed previously and that it is not already marked deleted, so that an already-deleted object is not flagged again on every subsequent revision.

Part 3: Test Cases

1. Initial Data Feeding

When a building is initially created, all objects should have an “added” status, provided all the necessary configuration has inputted in place and therefore passed the pre-validation gate.

PROJECT

Pilot Project 001

BUILDING

Tower A

REVISION

Rev 0

Objects

4 total

FLOOR	ZONE	SECTION	MATERIAL	STATUS
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added

Figure 3. Expected Change Status Outcome for Initial Data Feeding

2. Revision

When revision is triggered, the changed columns should turn to “modified” while the untouched objects are given status “unchanged”. Should there be any new objects inputted into the endpoint, they will receive an “added” status. The following example edits the first test case with 2 edited columns and 1 added column

PROJECTBUILDINGREVISION

Pilot Project 001Tower ARev 1

Objects5 total

OR	ZONE	SECTION	MATERIAL	STATUS
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Modified
und floor	Zone 1	C2 - Ø350 column	concrete K250	Modified
und floor	Zone 1	C3 - 300x300 column	concrete K250	Added

Figure 4. Expected Change Status Outcome for a Revision (Example: Addition and Modification)

3. Deletion

When an object is stated as deleted in interactive edit, or omitted from the bulk upload, the absent object should receive a deleted status for the current revision where the deletion is committed and should be cleared in the next one.

PROJECTBUILDINGREVISION

Pilot Project 001Tower ARev 2

Objects5 total

OR	ZONE	SECTION	MATERIAL	STATUS
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C3 - 300x300 column	concrete K250	Deleted

Figure 5. Expected Change Status Outcome for Deleted Object (Current Revision)

PROJECT

Pilot Project 001

BUILDING

Tower A

REVISION

Rev 3

Objects

4 total

OR	ZONE	SECTION	MATERIAL	STATUS
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged

Figure 6. Expected Change Status Outcome for Deleted Object (Next Revisions)

4. Restoration

Object restoration from deletion does not happen implicitly by reverting current data into the previous revision and instead considered a revision cycle itself. It is explicitly mentioned in the input (is_new=false) and automatically checked in the service model. The expected output should be an object with a “restored” status.

PROJECT

Pilot Project 001

BUILDING

Tower A

REVISION

Rev 4

Objects

5 total

OR	ZONE	SECTION	MATERIAL	STATUS
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C3 - 300x300 column	concrete K250	Restored

Figure 7. Expected Change Status Outcome for Restored Object

3. Validation Rules

The rules below are derived from the data dictionary, supplemented with additional validation that supports the pre-validation gates and revision workflow, grouped by the kind of check each one performs: uniqueness, append-only enforcement, referential existence, restricted value sets, and shape, format, and cardinality checks.

Table 5. Validation Rules

	Object / Field	Condition	Expected Result	Severity	Reasoning
Uniqueness					
VR01	<i>Floor.floor_name,</i> <i>Zone.zone_label</i>	Must be unique per building_id	Reject a duplicate name within the same building	Error	Two different buildings may each have their own “Level 2” or “Zone A” without conflict, since these labels only need to be locally meaningful.
VR02	<i>Project.project_name,</i> <i>Building.building_name,</i> <i>Section.sect_label,</i> <i>Material.mat_name,</i> <i>BarSpec.barspec_label</i>	Must be unique globally	Reject a duplicate value anywhere in the database	Error	These fields function as name-based lookup keys referenced from object input, so a name must resolve to exactly one row regardless of project or building.
VR03	<i>Grid.(building_id,</i> <i>grid_label, grid_axis)</i>	Must be unique as a combination	Reject a duplicate combination	Error	A label alone may repeat across axes (e.g. “1” on both X and Y), so uniqueness is only meaningful as a triple.
VR04	<i>Revision.(building_id,</i> <i>rev_number)</i>	Must be unique as a combination	Reject a duplicate rev_number within the same building	Error	Enforces that revisions remain strictly linear per building, preventing two revisions from claiming the same position in a building’s history.
VR05	<i>Zone.pour_seq</i>	Must be unique per building_id	Reject a duplicate sequence number within the same building	Error	Two zones in the same building cannot share the same pour order.
Project Configuration Append-Only Rules					
VR06	<i>Floor.elevation</i>	Append-only once referenced by an object	Reject an in-place edit; require a new row instead	Warning	Editing elevation directly would let an unchanged object silently shift position with no revision trail recording it.
VR07	<i>Material.mat_type,</i> <i>mat_strength, mat_weight</i>	Append-only once referenced	Reject an in-place edit; require a new row instead	Warning	Same reasoning as VR06 , applied to material properties an object may already depend on.
VR08	<i>BarSpec.barspec_dia,</i> <i>barspec_type,</i> <i>barspec_grade,</i> <i>barspec_weight</i>	Append-only once referenced	Reject an in-place edit; require a new row instead	Warning	Same reasoning as VR06 , applied to reinforcement specification properties.
VR09	<i>Section.dimension,</i> <i>Grid.grid_coord</i>	Append-only once referenced	Reject an in-place edit; require a new row instead	Warning	Follows directly from the snapshot principle: the old row is retained untouched, and a new row is created for any change.

Referential Existence					
VR10	<i>floor_name, zone_label</i> (Object input)	Must resolve to an existing row within the building given in the URL path	Reject the submission if not found in that building's scope	Error	These fields are building-scoped references, so existence must be checked against the correct building, not the whole database.
VR10b	<i>sect_label + obj_type, mat_name, barspec_label</i> (Object/Reinforcement input)	Must resolve to an existing row globally	Reject the submission if not found anywhere in the database	Error	These entities have no building scope, so existence is checked globally.
Enumeration / Value Restriction					
VR11	<i>Section.obj_type, Object.obj_type</i>	Must be one of: column, beam, wall, slab, footing, stair	Reject any other value	Error	Restricts member classification to the six types which the reinforcement and geometry rules are designed around.
VR12	<i>Object.change_status</i>	Must be one of: added, modified, unchanged, deleted, restored	Reject any other value	Error	Keeps the revision status vocabulary closed and consistent with the three-workflow design.
VR13	<i>Reinforcement.bar_role</i>	Must be one of: starter, longitudinal, transverse	Reject any other value	Error	Bar role determines which detailing rules apply (splice location, anchorage, termination), so an unrecognized role cannot be processed downstream.
VR14	<i>BarSpec.barspec_type</i>	Must be one of: deformed, plain	Reject any other value	Error	Matches the two bar surface types recognized by the reinforcement catalogue.
VR15	<i>Grid.grid_axis</i>	Must be one of: x, y	Reject any other value	Error	A grid line is either horizontal or vertical; no third axis is modeled.
Shape, Format & Cardinality					
VR16	<i>Section.dimension</i>	Structure must match its shape: {shape, width, depth} for rectangular; {shape, diameter} for circular; {thickness} for shell sections	Reject a mismatched or incomplete structure	Error	Prevents a section from carrying fields that belong to a different shape, which would otherwise silently corrupt the quantity calculation.
VR17	<i>Object.geometry_points</i>	Exactly two points for axis-point types (column, beam, footing); at least three points for boundary-point types (wall, slab, stair)	Reject an incorrect point count for the object's type	Error	An axis-point member is defined by its two endpoints; a boundary-point member needs at least three points to enclose an area.
VR18	<i>Reinforcement.bar_space, bar_hook_type</i>	Required when bar_role = 'transverse'; must be null otherwise	Reject a row that violates this pairing	Error	These fields only carry meaning for transverse (confinement) bars and are undefined for longitudinal or starter bars.
VR19	<i>Reinforcement.bar_count, bar_len</i>	Must be greater than 0	Reject a zero or negative value	Error	A reinforcement row with no bars or no length is not a physically valid entry.

VR20	<i>Object.reinforcements (array cardinality)</i>	At least one row required if the referenced material is concrete; zero rows valid if steel	Reject a concrete object with no reinforcement rows	Error	A concrete member without any reinforcement definition is almost always a data-entry omission, while a steel member legitimately has none.
VR-21	<i>Grid.grid_coord</i>	Must always be an explicit {"x": ..., "y": ...} object, regardless of axis	Reject an implicit scalar or array form	Error	Keeps the coordinate structure identical for both axes, so the frontend never needs conditional logic to plot a point.

Appendix A. Data Dictionary

Entity/ Field Name	Meaning	Data Type	Req.	Unit	Example	Source	Relationship	Validation Note
projects								
<i>project_id</i>	Database row identifier for project	varchar	Required	-	"PRJ-001"	System-Generated (Triggered by project creation by user)	Primary Key	Must be unique, therefore auto-generated by system based on the order of project created
<i>project_name</i>	User-facing project name,	varchar	Optional	-	"AA Apartment Complex"	Human Input	Unique	- Auto-filled with <i>project_id</i> as default when empty. - Freely editable, change to this field does not trigger new revision
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
buildings								
<i>building_id</i>	Database row identifier for building	varchar	Required	-	"BLD-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of building added
<i>building_name</i>	User-facing building name	varchar	Optional	-	"AA Complex - West Building"	Human Input	Unique	- Auto-filled with <i>project_id</i> as default when empty. - Freely editable, change to this field does not trigger new revision
<i>project_id</i>	Which project the building belongs to	varchar	Required	-	"PRJ-001"	System-Assigned (from URL path)	Foreign key to <i>projects.project_id</i>	- Value supplied by the POST path, not typed manually - Value must previously exist in the <i>projects</i> table
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
revisions								
<i>rev_id</i>	Database row identifier for building	varchar	Required	-	"REV-001"	System-Generated	Primary Key	- Must be unique, therefore auto-generated by system based on the order of revision uploaded or changed - Does not represent "the first revision" for a specific building but rather "revision data number one" for the global database
<i>building_id</i>	Which building the revision belongs to	varchar	Required	-	"BLD-001"	System-Assigned (from URL path)	Foreign Key to <i>buildings.building_id</i>	- Value supplied by the URL path, not typed manually - Value must previously exist in the <i>buildings</i> table
<i>rev_number</i>	Chronological number of the revision for a specific building, since <i>rev_id</i> belongs to global database and doesn't represent the order of revision	integer	Required	-	"0, 1, 2, 3, ..."	System-Assigned	Unique for one building	- Not unique, since multiple building can reside in the same table - First version of building is given number "0" - Unique for a single building

<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
floors								
<i>floor_id</i>	Database row identifier for floors	varchar	Required		"FL-001"	System-Generated	Primary Key	- Must be unique, therefore auto-generated by system based on the order of floor configuration added - Does not represent "the first revision" for a specific building but rather "revision data number one" for the global database
<i>floor_name</i>	User-facing floor name	varchar	Required		"LG", "Mezzanine", "1", "2", "3"	Human Input	Unique for one building	- Unique within a building, not globally (two different buildings may each have their own "Level 2") - Freely editable, change to this field does not trigger new revision - Required not as database concern but as human-error prevention, since floor_id does not represent floor order of a specified building
<i>elevation</i>	Elevation of floor	integer	Required	mm	"3000", "6000"	Human Input	None	Exist as data for generating drawing (currently out of scope), not as relation to member location
<i>building_id</i>	Which building the floor belongs to	varchar	Required		"BLD-001"	System-Assigned (from URL path)	Foreign Key to <i>buildings</i> . <i>building_id</i>	- Value supplied by the URL path, not typed manually - Value must previously exist in the <i>buildings</i> table
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
grid								
<i>grid_id</i>	Database row identifier for grids	varchar	Required	-	"G-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of grid configuration added
<i>building_id</i>	Which building the grid belongs to	varchar	Required		"BLD-001"	System-Assigned (from URL path)	Foreign Key to <i>buildings</i> . <i>building_id</i>	- Value supplied by the URL path, not typed manually - Value must previously exist in the <i>project_id</i> table
<i>grid_label</i>	User-facing grid line name	varchar	Required		"A", "B", "1", "2", "3"	Human Input	Unique together with <i>building_id</i> + <i>grid_axis</i>	- Non-unique, but the combination of (<i>building_id</i>, <i>grid_label</i>, <i>grid_axis</i>) must be unique - Freely editable, change to this field does not trigger new revision
<i>grid_axis</i>	Grid axis (whether horizontal line X or	varchar	Required		"X" and "Y"	Human Input	None	Restricted to ("X", "Y")

	vertical lines Y)							
<i>grid_coord</i>	Grid position	jsonb	Required	mm	"{"x": 0, "y": 6000}"	Human Input	None	Always an explicit {x, y} object regardless of <i>grid_axis</i>, never an implicit scalar or array form
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
zones								
<i>zone_id</i>	Database row identifier for zones	varchar	Required	-	"ZN-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of zone configuration added
<i>zone_label</i>	User-facing zone name	varchar	Required	-	"AA-01 West Wing"	Human Input	Unique for one building	- Unique within a building, not globally - Freely editable, change to this field does not trigger new revision - Required not as database concern but as engineering information, since <i>zone_id</i> does not represent zone order of a specified building
<i>pour_seq</i>	User-determined pour sequence for a specific zone	integer	Required	-	"1","2","3"	Human Input	None	Must be unique for the same building_id
<i>building_id</i>	Which building the zone belongs to	varchar	Required		"BLD-001"	System-Assigned (from URL path)	Foreign Key to <i>buildings</i> . <i>building_id</i>	- Value supplied by the URL path, not typed manually - Value must previously exist in the buildings table
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
materials								
<i>mat_id</i>	Database row identifier for materials	varchar	Required	-	"MAT-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of material configuration added
<i>mat_name</i>	User-facing material name	varchar	Required	-	"Beton fc'25"	Human Input	Unique	Required as engineering informatio, since <i>material_id</i> does not represent useful engineering information
<i>mat_type</i>	Explicit description of material type	varchar	Required	-	"concrete", "steel"	Human Input	None	-
<i>mat_strength</i>	Material strength detail	integer	Required	kg/cm2	"100","250"	Human Input	None	-
<i>mat_weight</i>	Material strength weight	integer	Required	kg/m3	"2400","7850"	Human Input	None	-
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
sections								
<i>sect_id</i>	Database row identifier for sections	varchar	Required	-	"SECT-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of section configuration added

<i>sect_label</i>	User-facing section name	varchar	Required	-	"B-1.250.500", "H-150"	Human Input	Unique	Required as engineering informatio, since sect_id does not represent useful engineering information
<i>obj_type</i>	Explicit description of object type (members and shells)	varchar	Required	-	"column", "beam", "slab", "wall"	Human Input	Composite Primary Key with sect_id	Restricted to ('column', 'beam', 'wall', 'slab', 'footing', 'stair')
<i>dimension</i>	Dimension for each object type, described in JSONB since dimension properties depend on the object type	jsonb	Required	-	{"shape": "rectangular", "width": 200, "depth": 400; {"thickness": 200}	Human Input	None	Shape-dependent structure: rectangular members use {shape, width, depth}; shells use {thickness}.
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
barspec								
<i>barspec_id</i>	Database row identifier for reinforcement catalogue	varchar	Required	-	"BAR-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of reinforcement catalogue added
<i>barspec_label</i>	User-facing rebar catalogue name	varchar	Required	-	"D16", "ø8"	Human Input	Unique	Required as engineering information, since barspec_id does not represent useful engineering information
<i>barspec_dia</i>	Bar diameter	integer	Required	mm	"16", "32"	Human Input	None	-
<i>barspec_type</i>	Bar type	varchar	Required		"deformed", "plain"	Human Input	None	Restricted to ('deformed', 'plain')
<i>barspec_grade</i>	Material grade detail for reinforcement	varchar	Required		"BJTS 40"	Human Input	None	-
<i>barspec_weight</i>	Material weight for reinforcement	integer	Required	kg/m3	"7850"	Human Input	None	-
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
identities								
<i>stable_id</i>	Internal stable identifier for a single member	varchar	Required	-	UUID string	System-Calculated (Generated through "is_new" bool input class)	Primary Key	Required as database design compliance, not used for engineering information
<i>ifc_global_id</i>	External IFC ID from BIM software	varchar	Optional	-	IFC ID	External Input	Unique	Could be used as is_new definition when present
<i>is_active</i>	Identifier for active status workflow	bool	Required		"true","false"	System-Calculated	None	-
<i>created_at</i>	Timestamp for historic data record	timestamp	Required	-	"2026-09-15 14:30:00"	System-Generated	None	-
objects								
<i>obj_id</i>	Database row identifier for objects	varchar	Required	-	"OBJ-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of objects added
<i>obj_mark</i>	User-facing object name	varchar	Required	-	"C1.FL02.001"	Human Input	None	Required as engineering information, since

								obj_id does not represent useful engineering information
stable_id	Which identifier the object is represented (revision-proof stable identity)	varchar	Required	-	UUID Value	System-Calculated	Foreing Key to identities. stable_id	Value must previously exist in the identities table
rev_id	Which revision the object input belongs to	varchar	Required	-	"REV-001"	System-Assigned	Foreing Key to revisions.rev_id	Value is autogenerated based on the order of revision created
change_status	Status sorter for object-level revision granularity	varchar	Required	-	"added", "modified", "unchanged", "deleted", "restored"	System-Calculated	None	Restricted to ('added', 'modified', 'unchanged', 'deleted', 'restored')
obj_type	Type of object	varchar	Required	-	"column", "beam", "slab", "wall"	Human Input	Composite FK with sect_label to sections.(sect_label, obj_type)	Restricted to ('column', 'beam', 'wall', 'slab', 'footing', 'stair')
floor_id	Which floor the object belongs to	varchar	Required	-	"FL-002"	Human Input (accepted as floor_name at the API layer, resolved to floor_id server-side)	Foreing Key to floors.floor_name	Input floor_name must previously exist within the building given in the URL path (D44/D45 scoped validation). Stored as floor_id, not the name itself, so that a later rename of the floor still resolves correctly for old objects (D27)
zone_id	Which zone the object belongs to	varchar	Required	-	"ZN-001"	Human Input (accepted as zone_label at the API layer, resolved to zone_id server-side)	Foreing Key to zones.zone_name	Input zone_label must previously exist within the building given in the URL path. Stored as zone_id, not the label itself, for the same reason as floor_id
sect_id	Which section the object references	varchar	Required	-	"SECT-001"	Human Input (accepted as sect_label + obj_type at the API layer, resolved to sect_id server-side)	Composite FK with obj_type to sections.(sect_label, obj_type)	Input sect_label must previously exist globally. Stored as sect_id, not the label itself
mat_id	Which material the object references	varchar	Required	-	"MAT-001"	Human Input (accepted as mat_name at the API layer, resolved to mat_id server-side)	Foreign Key to materials. mat_name	Input mat_name must previously exist globally. Stored as mat_id, not the name itself
geometry_points	Start and end point of a geometry	Jsonb	Required	mm	"[[0,0,0], [0,0,3000]]"	Human Input	None	- Member objects: must be two points - Shell objects: must atleast be three points
reinforcement								
bar_id	Database row identifier for a reinforcement group within an object	varchar	Required		"RBAR-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system based on the order of reinforcement rows added
obj_id	Which object this reinforcement group belongs to	varchar	Required		"OBJ-001"	System-Assigned	Foreign Key to objects.obj_id	Value must previously exist in the

								objects table
<i>barspec_label</i>	Reinforcement specification referenced by this bar group	varchar	Required		"D16"	Human Input	Foreign Key to barspec. barspec_label	- Input barspec_label must previously exist in the barspec table. - Stored as barspec_id, not the label itself
<i>bar_role</i>	Structural function of this bar group within the member	varchar	Required		"longitudinal", "transverse", "starter"	Human Input	None	Restricted to ('starter',, 'longitudinal',, 'transverse')
<i>bar_count</i>	Number of identical bars grouped in this row	integer	Required	-	"4", "20"	Human Input	None	Greater than 0
<i>bar_len</i>	Length of a single bar in this group	integer	Required	mm	"3500", "1600"	Human Input	None	Greater than 0
<i>bar_space</i>	Spacing between bars, applicable only to transverse bars	integer	Optional	mm	"150"	Human Input	None	Required when bar_role='transverse'; must be null otherwise
<i>bar_hook_type</i>	Hook/bend detail at the bar end, applicable only to transverse bars	varchar	Optional		"90"	Human Input	None	Required when bar_role='transverse'; must be null otherwise
quantity								
<i>qty_id</i>	Database row identifier for a quantity record	varchar	Required		"QTY-001"	System-Generated	Primary Key	Must be unique, therefore auto-generated by system
<i>obj_id</i>	Which object this quantity calculation belongs to	varchar	Required		"OBJ-001"	System-Assigned	Foreign Key to objects.obj_id	- Value must previously exist in the objects table - Exactly one quantity row per object row
<i>qty_sect</i>	Product of section volume and material weight	decimal (10,2)	Required	m3 or kg	"40 m3"; "1000kg"	System-Calculated	None	Shown directly in m3 if the object's material is concrete, kg if steel.
<i>qty_bar</i>	Product of reinforcement volume and bar weight	decimal (10,2)	Required	kg	"50 kg"	System-Calculated	None	Derived from total reinforcement volume (associated Reinforcement rows) multiplied by the referenced BarSpec's barspec_weight. Stored as decimal(10,2) for the same precision reason as qty_sect